

Relazione Progetto

Curriculum Learning per un Drone Antincendio
mediante Deep Reinforcement Learning

Federico Tagliabue
Matricola: 945103

Gregorio Musetti
Matricola: 947564

Indice

1	Inquadramento del problema	3
2	Fondamenti teorici	4
2.1	Reinforcement Learning	4
2.2	Q-Learning e Deep Q-Network	4
2.3	Experience Replay	5
2.4	Target Network	5
2.5	Reward Shaping	6
2.6	Curriculum Learning	6
3	Ambiente e implementazione	7
3.1	Struttura dell'ambiente	7
3.2	Rappresentazione dello stato	7
3.3	Architettura della rete neurale	8
3.4	Funzione di reward	8
3.5	Curriculum Learning e struttura dei livelli	10
3.6	Metriche di valutazione	11
4	Sistema attuale e risultati	12
4.1	Come avviare il training	12
4.2	Visualizzazione dell'agente addestrato	13
4.3	Analisi dei risultati	13
4.3.1	Reward media — Confronto livelli 1a, 1b, 1c	13
4.3.2	Success rate — Confronto livelli 1a, 1b, 1c	14
4.3.3	Step medi — Confronto livelli 1a, 1b, 1c	15
4.3.4	Loss MSE — Confronto livelli 1a, 1b, 1c	16
4.3.5	Distribuzione degli esiti — Confronto livelli 1a, 1b, 1c	17
4.4	Test di generalizzazione	18
4.4.1	Success rate	19
4.4.2	Steps per episodio	20
4.4.3	Distribuzione degli esiti	21
5	Problema del Multi-Agent	22
5.1	Estensione al caso multi-agent	22
5.2	Risultati e limiti osservati	22
6	Test e modifiche sperimentali	25
6.1	Modifiche alla rappresentazione dello stato	25

7	Miglioramenti futuri	28
7.1	Coordinamento multi-agent con QMIX	28
7.2	Divisione dei fuochi tramite algoritmi di assegnazione	28
7.3	Ottimizzazione del percorso	29
7.4	Revisione e ottimizzazione dei livelli di curriculum	29
7.5	Aumento del livello di astrazione dell'ambiente	29

Capitolo 1

Inquadramento del problema

Il seguente progetto si colloca nell'ambito del Reinforcement Learning applicato a sistemi autonomi. In particolare, l'obiettivo è studiare il comportamento di un singolo drone che opera in un ambiente simulato, nel quale deve apprendere autonomamente come individuare ed estinguere incendi distribuiti all'interno di una griglia bidimensionale.

I droni possono essere utilizzati in numerosi contesti applicativi, in cui gli agenti devono operare in ambienti parzialmente sconosciuti prendendo decisioni in tempo reale sulla base delle informazioni disponibili.

Tra le possibili applicazioni, quella che siamo andati a studiare riguarda la gestione di incendi. In presenza di uno o più focolai, il drone deve essere in grado di esplorare l'ambiente, individuare le aree da raggiungere e prendere decisioni efficienti per intervenire in maniera efficace. La complessità del problema aumenta ulteriormente quando sono presenti ostacoli, vincoli di movimento e la necessità di evitare collisioni in un contesto multi-agent.

L'obiettivo del progetto è quindi progettare e addestrare un singolo agente in grado di affrontare il problema dell'estinzione di incendi in un ambiente simulato, valutando l'efficacia delle tecniche di Reinforcement Learning e delle strategie di apprendimento progressivo.

Capitolo 2

Fondamenti teorici

2.1 Reinforcement Learning

Il **Reinforcement Learning (RL)** è una tecnica di apprendimento automatico nel quale un agente apprende un comportamento ottimale attraverso l'interazione con un ambiente. Ad ogni istante l'agente osserva lo stato corrente del drone, seleziona un'azione ed ottiene una ricompensa che rappresenta una misura della qualità della decisione presa.

L'obiettivo dell'agente consiste nel massimizzare la ricompensa ottenuta nel corso dell'interazione, apprendendo progressivamente una politica decisionale in grado di associare ad ogni stato l'azione più vantaggiosa. L'agente deve quindi esplorare autonomamente l'ambiente e valutare le conseguenze delle proprie azioni.

2.2 Q-Learning e Deep Q-Network

Uno degli approcci più diffusi nel Reinforcement Learning è il **Q-Learning**, un algoritmo che apprende una funzione detta *Q-function*.

La funzione Q associa ad ogni coppia stato-azione un valore che rappresenta la qualità di una determinata azione in uno specifico stato e seguendo successivamente la politica appresa.

Durante l'apprendimento l'agente aggiorna progressivamente la stima della funzione Q fino a convergere verso una politica che massimizza la reward.

Il principale limite del Q-Learning classico riguarda la difficoltà di rappresentare esplicitamente tutti gli stati possibili quando lo spazio degli stati diventa molto grande. In questi casi è necessario utilizzare tecniche di approssimazione più avanzate.

Per affrontare problemi caratterizzati da spazi di stato elevati è possibile sostituire la classica tabella Q con una rete neurale artificiale. Questo approccio prende il nome di **Deep Q-Network (DQN)** [1].

L'idea consiste nell'utilizzare una rete neurale per approssimare la funzione Q, ricevendo in ingresso una rappresentazione dello stato dell'ambiente e producendo in uscita una stima del valore associato a ciascuna azione disponibile.

L'utilizzo delle reti neurali consente di generalizzare la conoscenza acquisita durante l'addestramento. Tuttavia l'apprendimento tramite reti neurali può diventare instabile perché le esperienze sono spesso simili tra loro nel tempo e i valori usati come riferimento cambiano continuamente. Per ridurre questi problemi si usano tecniche come Experience Replay e Target Network.

2.3 Experience Replay

L'**Experience Replay** è una tecnica utilizzata per migliorare la stabilità dell'addestramento dei Deep Q-Network [1].

Durante l'interazione con l'ambiente ogni esperienza viene memorizzata sotto forma della quintupla

$$(s, a, r, s', d)$$

dove

- s rappresenta lo stato corrente;
- a rappresenta l'azione eseguita;
- r rappresenta la ricompensa ottenuta;
- s' rappresenta lo stato successivo;
- d flag booleano che indica se lo stato s' è terminale, ovvero se l'episodio si è concluso.

Le esperienze vengono salvate all'interno di una memoria detta *Replay Buffer* e successivamente campionate in maniera casuale durante la fase di addestramento.

Questo approccio impedisce all'agente di imparare sempre da esperienze molto simili, dandogli la possibilità di riutilizzare anche esperienze passate, e rende l'apprendimento più veloce e stabile. Il buffer ha una capacità massima di 10.000 esperienze per drone ed è implementato come una lista che sovrascrive le esperienze più vecchie.

Quando il buffer contiene almeno 64 esperienze, viene estratto casualmente un campione di quella dimensione e utilizzato per aggiornare i pesi della rete. La loss è calcolata come errore quadratico medio tra il valore Q corrente e il target calcolato secondo la regola di Bellman con fattore di sconto $\gamma = 0.9$.

Il valore target viene quindi calcolato come

$$Target = r + \gamma \max_a Q_{target}(s', a)(1 - done).$$

Per l'ottimizzazione dei parametri della rete viene utilizzato l'algoritmo **Adam** [2] con un learning rate pari a

$$10^{-4}$$

2.4 Target Network

Un ulteriore problema dei DQN è rappresentato dal fatto che la stessa rete viene utilizzata sia per stimare i valori correnti sia per generare i valori target utilizzati durante l'aggiornamento.

Per risolvere il problema dell'oscillazione dei parametri viene introdotta una seconda rete neurale, detta **Target Network** [4], che mantiene una copia dei parametri della rete principale.

La rete target viene aggiornata periodicamente copiando i pesi della rete principale, fornendo così valori di riferimento più stabili nel calcolo della loss.

Durante calcolo, la rete principale fornisce il valore $Q(s, a)$ corrispondente all'azione effettivamente eseguita, mentre la rete target fornisce il valore $\max_{a'} Q(s', a')$, ovvero il Q-value massimo tra le azioni disponibili nello stato successivo.

La loss viene quindi calcolata come l'errore quadratico medio tra questi due valori, dove il target è definito come $r + \gamma \cdot \max_{a'} Q(s', a') \cdot (1 - d)$.

2.5 Reward Shaping

Nonostante queste tecniche consentano di rendere più robusto l'algoritmo, la velocità e l'efficacia dell'apprendimento dipendono fortemente dalla definizione della **funzione di reward**. In molti problemi reali, infatti, le ricompense significative vengono ricevute soltanto al raggiungimento dell'obiettivo finale rendendo difficile per l'agente comprendere quali azioni abbiano effettivamente contribuito al successo dell'episodio.

Per questo motivo viene utilizzato il **reward shaping**, una tecnica che introduce ricompense intermedie con lo scopo di guidare l'agente verso il comportamento desiderato.

Nel progetto è stato adottato il reward shaping basato sulla variazione della distanza di Manhattan dal fuoco più vicino: il drone riceve una ricompensa positiva proporzionale all'avvicinamento al fuoco e una penalità in caso di allontanamento, come descritto in dettaglio nella sezione dedicata alla funzione di reward 3.4.

2.6 Curriculum Learning

Anche in presenza di una funzione di reward ben progettata, l'addestramento può risultare particolarmente complesso quando l'ambiente presenta numerose situazioni differenti o richiede l'acquisizione simultanea di molte competenze.

Per questo motivo viene spesso utilizzata una metodologia chiamata **Curriculum Learning**. L'idea alla base è quella di consentire all'agente di acquisire inizialmente competenze in ambienti semplici e successivamente trasferirle a scenari più complessi.

Questo approccio può migliorare la velocità di convergenza, la stabilità dell'apprendimento e la capacità di affrontare problemi particolarmente difficili.

In molti casi il Curriculum Learning viene combinato con tecniche di **transfer learning**, nelle quali i parametri appresi in un compito vengono utilizzati come punto di partenza per il successivo addestramento.

Capitolo 3

Ambiente e implementazione

3.1 Struttura dell'ambiente

L'ambiente di simulazione è rappresentato da una griglia bidimensionale quadrata.

All'interno di questa griglia operano tre tipologie di entità: il drone, i fuochi e gli ostacoli.

Il **drone** è l'agente attivo del sistema. Ad ogni passo temporale può eseguire una tra quattro azioni disponibili: muoversi verso sinistra, destra, su o giù di una cella.

I **fuochi** rappresentano gli obiettivi che l'agente deve raggiungere e spegnere: quando il drone si posiziona sulla stessa cella di un fuoco, quest'ultimo viene rimosso dall'ambiente.

Gli **ostacoli** sono celle statiche che il drone non può attraversare.

All'inizio di ogni episodio tutte le entità vengono posizionate casualmente in celle distinte della griglia, garantendo che nessuna sovrapposizione si verifichi. L'episodio termina in quattro casi:

- il drone esce dai confini della griglia;
- il drone si scontra con un ostacolo;
- tutti i fuochi vengono spenti;
- viene superato il numero massimo di passi consentiti (`max_steps`) [5].

I primi due casi rappresentano un fallimento, il terzo un successo e l'ultimo un timeout.

L'implementazione dell'ambiente è contenuta nella classe 'DroneGame', che gestisce lo stato delle entità, l'avanzamento temporale e il calcolo della reward ad ogni step. La classe supporta sia la modalità single-agent sia quella multi-agent, con un numero di droni configurabile da uno a tre.

3.2 Rappresentazione dello stato

Ad ogni passo temporale il drone osserva l'ambiente attraverso un vettore di stato composto da 22 valori numerici. La dimensione del vettore rimane costante per tutti i livelli del curriculum, permettendo il trasferimento diretto dei pesi della rete neurale durante il passaggio ai livelli successivi.

Il vettore di stato è organizzato come segue:

- **Posizioni relative degli altri droni** (indici 0–3): due slot da 2 valori ciascuno, che rappresentano la posizione relativa degli altri due droni rispetto al drone corrente. Se un slot non è occupato da nessun drone, viene impostato con il valore -1.
- **Posizioni relative dei fuochi** (indici 4–13): cinque slot da 2 valori ciascuno, che rappresentano la posizione relativa di ogni fuoco rispetto al drone. Anche in questo caso gli slot corrispondenti a fuochi già spenti vengono impostati a -1.
- **Sensori di prossimità** (indici 14–17): quattro valori binari che indicano se la cella adiacente in ciascuna delle quattro direzioni è fuori dai confini della griglia o occupata da un ostacolo.
- **Contatori di visita** (indici 18–21): quattro valori estratti da una matrice $gridsize \times gridsize$ mantenuta per ogni drone, che tiene il conto di quante volte ciascuna cella è stata visitata nell’episodio corrente e viene azzerata ad ogni reset. I valori inseriti nello stato corrispondono alle quattro celle adiacenti, normalizzati rispetto al numero massimo di passi consentiti.

3.3 Architettura della rete neurale

Per approssimare la funzione Q utilizziamo una rete neurale totalmente connessa, definita nella classe `actor`.

La rete riceve in ingresso il vettore di stato e attraversa quattro layer lineari: viene applicata un’attivazione ReLU dopo ciascuno dei primi tre layer, mentre il quarto layer di uscita è lineare, in modo da produrre quattro valori Q non vincolati, uno per ciascuna azione disponibile.

La struttura della rete è la seguente: il layer di ingresso riceve il vettore di stato da 22 valori e lo proietta in uno spazio di 128 neuroni.

I due layer intermedi mantengono la stessa dimensione, mentre il layer di uscita produce quattro valori, uno per ciascuna delle azioni disponibili. L’azione selezionata corrisponde all’indice con valore Q massimo.

Per stabilizzare l’addestramento viene impiegata una **target network**, ovvero una seconda copia della rete i cui pesi vengono aggiornati periodicamente copiando quelli della rete principale ogni 500 passi.

Questa separazione impedisce che i valori target utilizzati nel calcolo della loss cambino ad ogni iterazione, riducendo le oscillazioni durante l’apprendimento.

3.4 Funzione di reward

La funzione di reward è stata progettata con l’obiettivo di guidare progressivamente l’agente verso il comportamento desiderato.

Dopo diversi test e considerazioni, i valori utilizzati configurati nella struttura `RewardConfig` sono definiti come segue:

- **Reward per fuoco spento:** +100. Viene assegnata a tutti i droni quando uno di essi raggiunge un fuoco e lo estingue.
- **Bonus di completamento:** +200. Viene assegnato quando tutti i fuochi presenti nell’ambiente sono stati spenti.

- **Penalità per collisione con bordo o ostacolo:** -100 . Tale evento provoca la terminazione immediata dell'episodio [3].
- **Penalità per collisione tra droni:** -30 . La penalità viene assegnata ai droni coinvolti senza interrompere l'episodio [7].
- **Penalità per passo neutro:** -0.1 ad ogni passo in cui non viene compiuta alcuna azione significativa. Questa penalità incentiva il raggiungimento dell'obiettivo nel minor numero possibile di mosse [3].
- **Penalità per timeout:** -50 allo scadere del numero massimo di passi consentiti.

Oltre a questi segnali di base, vengono utilizzate due tecniche di reward shaping.

La prima è un **bonus di avvicinamento** proporzionale alla riduzione della distanza tra il drone e il fuoco più vicino [4] : ad ogni passo la distanza dal fuoco più vicino viene misurata prima di eseguire il movimento, confrontata con quella rilevata dopo e il drone riceve una reward aggiuntiva pari a questa differenza. Questo segnale guida l'agente nelle fasi iniziali dell'addestramento, quando le reward positive sono molto rare.

La seconda tecnica è una **penalità incrementale per revisita** [3]: ogni volta che un drone torna su una cella già visitata nella stessa sessione riceve una penalità pari a

$$-0.2 \times n,$$

moltiplicata per il numero di visite precedenti sulla stessa cella.

I contatori di visita utilizzati per calcolare questa penalità sono gli stessi esposti nello stato del drone (sezione 3.2): la stessa matrice serve quindi sia a costruire l'osservazione dell'agente sia a calcolare la reward.

Questa penalità è stata introdotta per eliminare un comportamento ciclico osservato durante le prime fasi di addestramento: in prossimità di ostacoli o bordi il drone tendeva a oscillare sulla stessa coppia di celle, avanzando verso l'ostacolo, percependo il pericolo e retrocedendo, per poi ripetere il ciclo indefinitamente. La penalità incrementale rende questo comportamento progressivamente più costoso, spingendo l'agente a esplorare percorsi alternativi.

Politica di esplorazione

Uno degli aspetti fondamentali del Reinforcement Learning riguarda il compromesso tra *exploration* ed *exploitation*.

L'esplorazione consiste nel provare nuove azioni al fine di raccogliere informazioni sull'ambiente, mentre l'exploitation sfrutta la conoscenza già acquisita per selezionare l'azione ritenuta migliore.

La politica di esplorazione che abbiamo utilizzato è ϵ -**greedy**. All'inizio di ogni livello viene impostato

$$\epsilon = 1,$$

ovvero il drone agisce in modo completamente casuale. Ad ogni episodio ϵ viene moltiplicato per

$$0.995,$$

fino a raggiungere il valore minimo di

$$\varepsilon_{\min} = 0.05.$$

che garantisce una minima esplorazione randomica anche nelle fasi avanzate dell'addestramento.

Il modello migliore per ciascun livello viene salvato automaticamente quando la reward media sugli ultimi 100 episodi supera il massimo storico. Vengono salvati anche checkpoint ogni 1000 episodi e un file `metrics.json` con l'intero storico delle metriche.

3.5 Curriculum Learning e struttura dei livelli

L'addestramento dell'agente è organizzato secondo una strategia di **Curriculum Learning**, nella quale il problema viene suddiviso in una successione di livelli a difficoltà crescente.

I livelli sono definiti nella struttura `LEVELS` del modulo `CurriculumConfig` e sono organizzati in tre fasi principali.

Prima fase

La **prima fase** (livelli 1a, 1b, 1c) prevede un singolo drone che opera in griglie di dimensione crescente (12×12 , 14×14 , 16×16) con un numero di fuochi e ostacoli che aumenta progressivamente.

L'obiettivo è che il drone acquisisca le competenze di navigazione di base: evitare i bordi e gli ostacoli, e raggiungere i fuochi in modo efficiente.

Seconda fase

La **seconda fase** (livelli 2a, 2b) introduce un secondo drone nell'ambiente.

Qui l'agente deve imparare a coordinarsi con un altro agente, evitando collisioni drone-drone e imparando a gestire situazioni in cui più fuochi devono essere raggiunti contemporaneamente.

Terza fase

La **terza fase** (livelli 3a, 3b) porta il numero di droni a tre e porta le dimensioni dell'ambiente al massimo previsto, con una griglia 18×18 , cinque fuochi e otto ostacoli.

In questa fase la complessità del coordinamento è massima.

Livello	Griglia	Fuochi	Ostacoli	Droni	Max passi	Episodi
1a	12×12	2	5	1	100	3500
1b	14×14	3	10	1	100	3500
1c	16×16	4	15	1	120	4000
2a	13×13	3	4	2	400	5000
2b	15×15	4	6	2	500	6000
3a	15×15	4	6	3	600	5000
3b	18×18	5	8	3	700	5000

Tabella 3.1: Configurazione dei livelli del Curriculum Learning.

Il passaggio da un livello al successivo avviene tramite **Transfer Learning**: i pesi del modello migliore del livello precedente (salvati nel file `best.pt`) vengono caricati come punto di partenza del livello successivo.

In questo modo le competenze acquisite in ambienti più semplici non vengono perse, ma vengono raffinate e adattate a scenari più complessi.

3.6 Metriche di valutazione

Per monitorare l'evoluzione dell'apprendimento vengono registrate diverse metriche durante tutto il processo di addestramento.

I valori visualizzati vengono aggregati sugli ultimi 100 episodi, e sono:

- **Reward media**: rappresenta la media delle reward cumulative ottenute durante un episodio, calcolata come media tra tutti i droni attivi.
- **Success rate**: indica la percentuale di episodi terminati con l'estinzione completa di tutti i fuochi presenti nell'ambiente.
- **Distribuzione degli esiti**: misura la percentuale di episodi conclusi per successo, timeout, collisione con il bordo oppure collisione con un ostacolo.
- **Numero medio di passi**: rappresenta il numero medio di azioni eseguite in ciascun episodio e costituisce un indicatore dell'efficienza della politica appresa.
- **Loss media**: descrive l'andamento dell'errore di approssimazione della funzione Q durante il processo di addestramento.
- **Collisioni drone-drone**: indica il numero totale di collisioni tra agenti nella modalità multi-agent e costituisce un indicatore del livello di coordinamento raggiunto.

Al termine dell'addestramento di ogni livello tutte le metriche vengono salvate nel file `metrics.json`.

Capitolo 4

Sistema attuale e risultati

4.1 Come avviare il training

Il training può essere eseguito in due modalità: per singolo livello o per l'intera sequenza del curriculum.

Per addestrare un singolo livello è sufficiente eseguire il relativo script dedicato. Ad esempio, per il livello 1a:

```
python train_level1a.py
```

Sono disponibili script analoghi per ciascuno degli altri livelli (`train_level1b.py`, `train_level1c.py`, ecc.). Ogni script stampa a schermo un riepilogo della configurazione del livello (dimensione griglia, numero di droni, fuochi, ostacoli e numero di episodi) e aggiorna il terminale ogni 200 episodi con le metriche correnti.

Per eseguire l'intera sequenza del curriculum in modo automatico è invece disponibile lo script `train_all.py`.

Questo script esegue i livelli in sequenza

$$1a \rightarrow 1b \rightarrow 1c \rightarrow 2a \rightarrow 2b \rightarrow 3a \rightarrow 3b$$

caricando automaticamente il modello migliore del livello precedente prima di iniziare quello successivo. Al termine di ogni livello viene stampato il tempo impiegato, e al termine dell'intera sequenza viene mostrato il tempo totale.

Al termine di ogni livello il sistema produce tre tipologie di file nella cartella

`Models/Level<id>/`

- `best.pt` contenente pesi del modello con la reward media più alta sugli ultimi 100 episodi;
- `checkpoint_<episodio>.pt` ovvero i checkpoint salvati ogni 1000 episodi;
- `metrics.json`, il file JSON con l'intero storico delle metriche per episodio.

4.2 Visualizzazione dell'agente addestrato

Per visualizzare graficamente il comportamento dell'agente a training completato è disponibile lo script `DronePlay.py`, che apre una finestra pygame e mostra l'agente operare in tempo reale nell'ambiente simulato.

Il comando base è il seguente:

```
python DronePlay.py --model_path Models/Level1a/best.pt --level 1a
```

I parametri disponibili sono:

- `--model_path` (obbligatorio): percorso al file `.pt` del modello da caricare.
- `--level`: livello del curriculum da usare come ambiente (default: `1a`).
- `--fps`: velocità di visualizzazione in frame per secondo (default: `10`).

Durante la visualizzazione la finestra mostra la griglia con uno o più droni, nel caso multi agent (colorati di blu, verde e rosso rispettivamente), i fuochi (arancione) e gli ostacoli (grigio). Un casella di testo riporta il numero di step corrente, i fuochi rimasti, l'ultima azione e l'ultima reward per ciascun drone. Al termine di ogni episodio il sistema attende un secondo e reimposta automaticamente l'ambiente.

Inizialmente l'agente opera in modalità completamente greedy, senza esplorazione: ad ogni step seleziona l'azione con il valore Q più alto prodotto dalla rete.

4.3 Analisi dei risultati

4.3.1 Reward media — Confronto livelli 1a, 1b, 1c

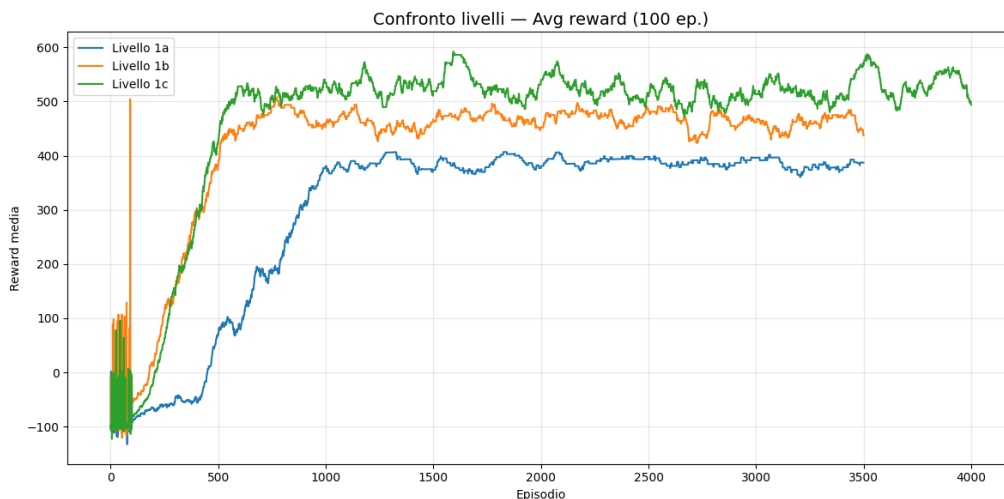


Figura 4.1: Confronto reward media.

Il grafico mostra l'andamento della reward media sugli ultimi 100 episodi per i tre livelli della fase single-agent, sovrapposti sullo stesso asse per evidenziare l'effetto del transfer learning. All'inizio i modelli mostrano scarse prestazioni a causa del valore di epsilon che fa muovere i droni casualmente per un'iniziale esplorazione della griglia.

Il dato più evidente riguarda la velocità di convergenza. Il livello 1a (blu) parte da valori negativi intorno a -100 e impiega circa 1.200 episodi per stabilizzarsi attorno a 380–400. I livelli 1b (arancione) e 1c (verde) invece raggiungono valori stabili già entro i primi 800 episodi grazie al transfer learning. I pesi caricati dal livello precedente forniscono un punto di partenza già competente, riducendo drasticamente la fase di esplorazione iniziale.

La maggiore oscillazione del livello 1c riflette la maggiore variabilità degli episodi su una griglia 16×16 con 4 fuochi e 15 ostacoli, dove la disposizione casuale delle entità influisce significativamente sull'esito.

4.3.2 Success rate — Confronto livelli 1a, 1b, 1c

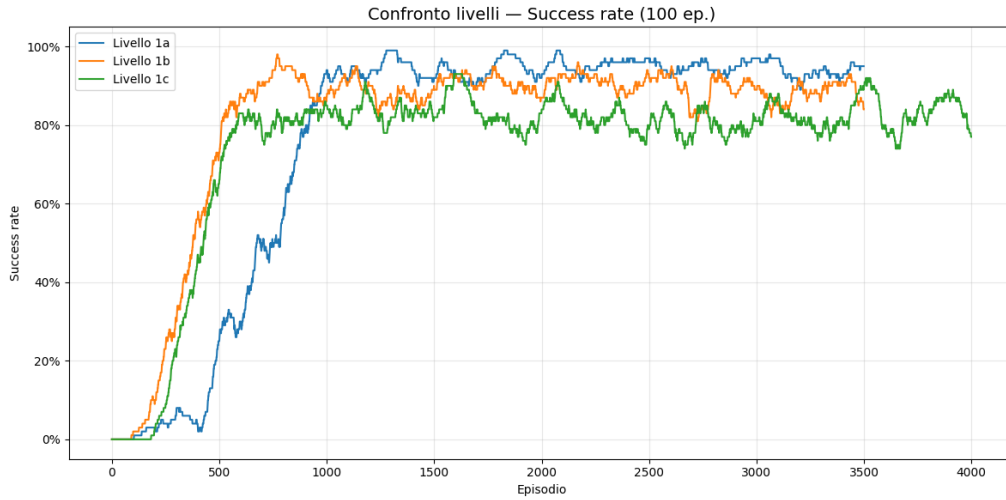


Figura 4.2: Confronto success rate.

Dal grafico si può valutare che i livelli 1b e 1c convergono quasi in parallelo e molto rapidamente, raggiungendo l'80% di successo già intorno all'episodio 600. Il livello 1a invece impiega quasi il doppio degli episodi (circa 900) per raggiungere lo stesso valore. Questo conferma l'efficacia del transfer learning: i pesi ereditati non migliorano solo la reward numerica, ma accelerano la capacità del drone di completare il compito.

Contrariamente a quanto ci si potrebbe aspettare, il livello 1a (blu) mostra il success rate più alto e più stabile nella fase finale, oscillando tra il 90% e il 95%. I livelli 1b e 1c sono più bassi e più rumorosi, con 1c che scende frequentemente verso il 75–80%. Questo può essere spiegato dalla difficoltà crescente dei livelli: su una griglia 16×16 con 15 ostacoli la probabilità di collisione accidentale è più alta e una parte degli episodi falliti dipende dalla configurazione casuale dell'ambiente.

Da notare che su tutti e tre i livelli il success rate non scende mai a zero neanche nelle primissime fasi. Questo significa che anche con $\varepsilon = 1.0$ (azioni completamente casuali) il drone riesce occasionalmente a spegnere tutti i fuochi, il che è plausibile su griglie relativamente piccole.

4.3.3 Step medi — Confronto livelli 1a, 1b, 1c

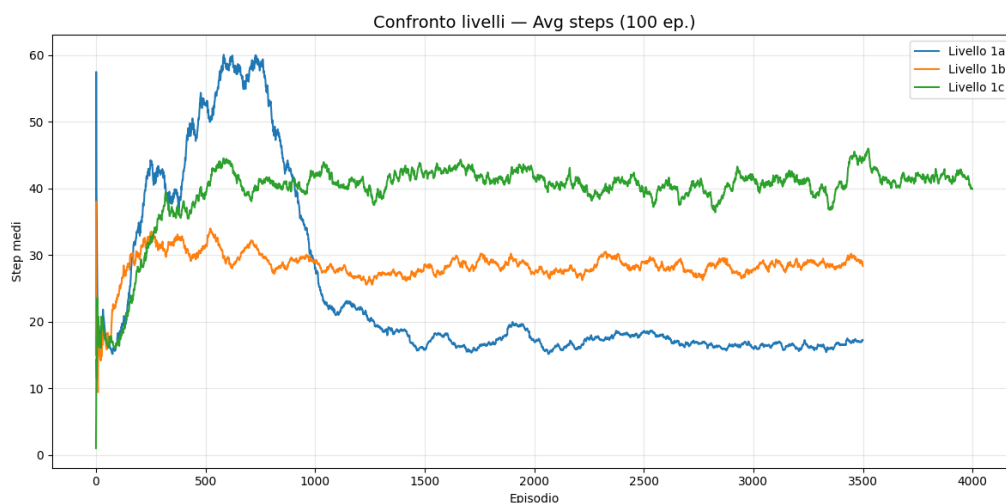


Figura 4.3: Confronto step medi.

In questo caso una curva decrescente indica miglioramento, in quanto il drone raggiunge l'obiettivo in meno passi.

Nella fase iniziale gli step medi del livello 1a salgono fino a circa 60, raggiungendo il picco intorno all'episodio 800, per poi scendere bruscamente fino a stabilizzarsi attorno a 17 dopo l'episodio 1000. Questo andamento è tipico dell'apprendimento DQN: nelle prime fasi il drone inizia a evitare le collisioni sopravvivendo più a lungo per episodio, ma non sa ancora dove andare e vaga per la griglia accumulando passi. Solo quando i pesi della rete neurale si aggiornano, raggiungendo risultati accettabili, il drone inizia a dirigersi attivamente verso i fuochi gli step crollano. Il picco a 60 non è quindi un peggioramento, ma una fase intermedia necessaria in cui il drone ha imparato a non morire ma non ancora a vincere.

I livelli 1b e 1c non mostrano il picco di 1a grazie al transfer learning: il drone parte già con una politica funzionante e gli step si stabilizzano quasi subito, rispettivamente intorno a 30 per 1b e 40 per 1c. Inoltre dopo aver raggiunto la convergenza il livello 1c presenta un numero medio di step più elevato rispetto ai livelli 1a e 1b, in quanto la dimensione della griglia è di più grandi dimensioni (16×16).

4.3.4 Loss MSE — Confronto livelli 1a, 1b, 1c

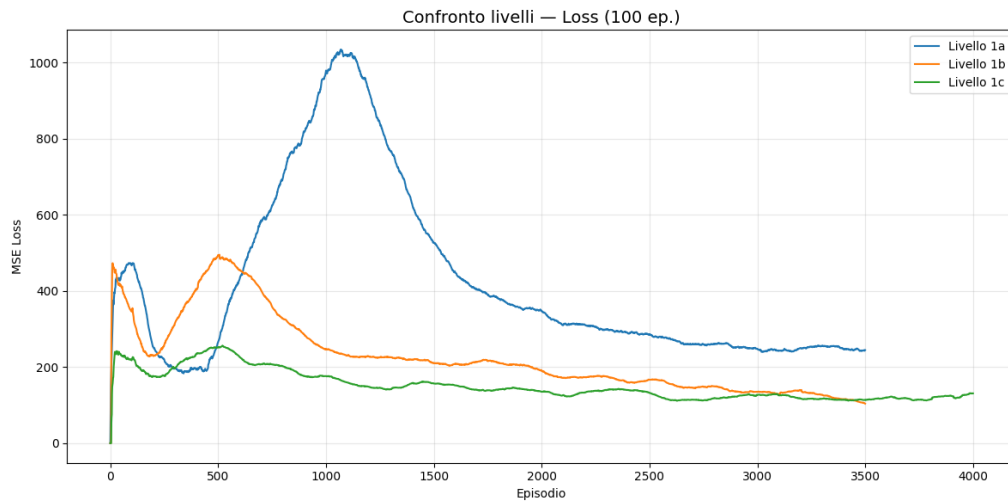


Figura 4.4: Confronto loss.

Il livello 1a (blu) mostra il comportamento più estremo: la loss parte intorno a 380, scende brevemente, poi risale fino a un picco di circa 1000 attorno all'episodio 1200, per poi decrescere lentamente e stabilizzarsi intorno a 280 a fine training. Questo picco coincide esattamente con la fase di transizione osservata nel grafico degli step medi, in cui il drone passa dall'esplorazione casuale all'exploitation della politica appresa. Durante questa fase la rete sta aggiornando i propri pesi e i valori Q cambiano rapidamente perché il comportamento dell'agente cambia frequentemente, e questo si traduce in una loss elevata.

I livelli 1b e 1c mostrano picchi molto più contenuti (rispettivamente circa 480 e 220). Questo è di nuovo una conseguenza del transfer learning in cui la rete parte già in uno stato vicino alla soluzione. Un dettaglio rilevante è che i valori finali di loss di 1b e 1c convergono allo stesso livello (intorno a 100–130), nonostante 1c sia un problema più difficile, il che suggerisce che entrambi i modelli hanno raggiunto un livello di approssimazione della funzione Q simile in termini di errore residuo.

4.3.5 Distribuzione degli esiti — Confronto livelli 1a, 1b, 1c

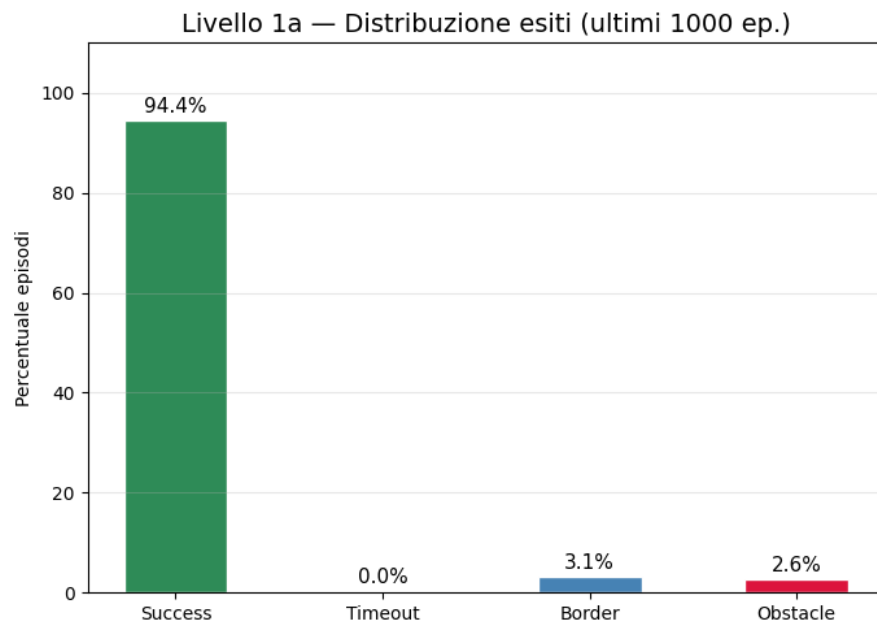


Figura 4.5: Distribuzione esiti livello 1a.

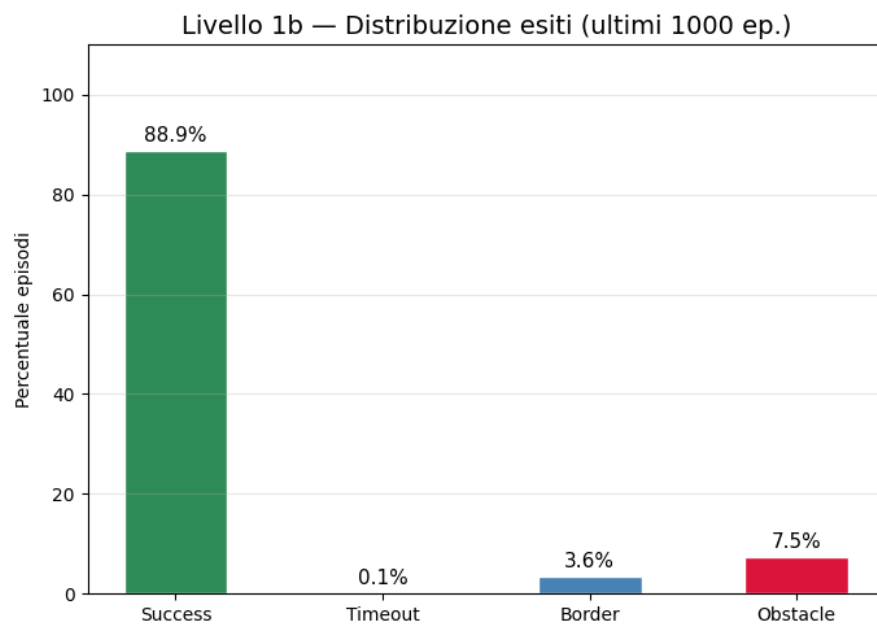


Figura 4.6: Distribuzione esiti livello 1b.

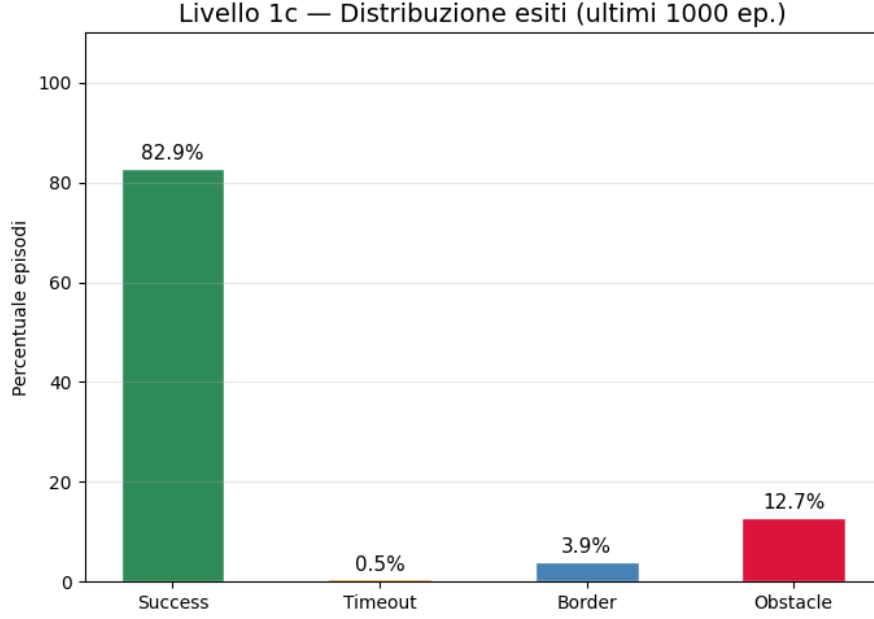


Figura 4.7: Distribuzione esiti livello 1c.

Esito	Livello 1a	Livello 1b	Livello 1c
Success	94.4%	88.9%	82.9%
Obstacle	2.6%	7.5%	12.7%
Border	3.1%	3.6%	3.9%
Timeout	0.0%	0.1%	0.5%

I grafici mostrano un comportamento coerente rispetto alle aspettative. Il **success rate rimane alto su tutti e tre i livelli**: 94.4% in 1a, 88.9% in 1b, 82.9% in 1c. Il calo progressivo riflette l'aumento di difficoltà del curriculum.

Il **timeout** rimane praticamente nullo: 0.0% in 1a, 0.1% in 1b, 0.5% in 1c.

Le **collisioni con ostacoli** crescono passando da 2.6% a 7.5% a 12.7%, riflettendo l'aumento progressivo di ostacoli nei tre livelli (5, 10 e 15 rispettivamente).

Le **collisioni con il bordo** rimangono invece quasi costanti tra i tre livelli (3.1%, 3.6%, 3.9%), a conferma che la capacità di evitare i confini della griglia viene acquisita in modo robusto già dal livello 1a e non viene intaccata dall'aumento di complessità.

4.4 Test di generalizzazione

Una volta completato l'addestramento della fase single-agent, è stato condotto un test di generalizzazione per verificare la capacità del modello di operare correttamente in un ambiente mai visto durante il training.

L'obiettivo era valutare se le competenze acquisite nei tre livelli del curriculum fossero generalizzate o semplicemente adattate alle configurazioni specifiche su cui il modello era stato addestrato.

Il livello di test, denominato *1test*, è stato progettato con parametri diversi rispetto a tutti i livelli del curriculum, aumentando il livello di complessità:

Parametro	Valore
Griglia	17×17
Fuochi	5
Ostacoli	20
Droni	1
Max passi	150
Episodi	3 000

Tabella 4.1: Parametri utilizzati per il test

Il modello caricato per questo test è ‘best.pt’ del livello 1c, l’ultimo livello della fase single-agent, senza alcun addestramento sul nuovo ambiente.

4.4.1 Success rate

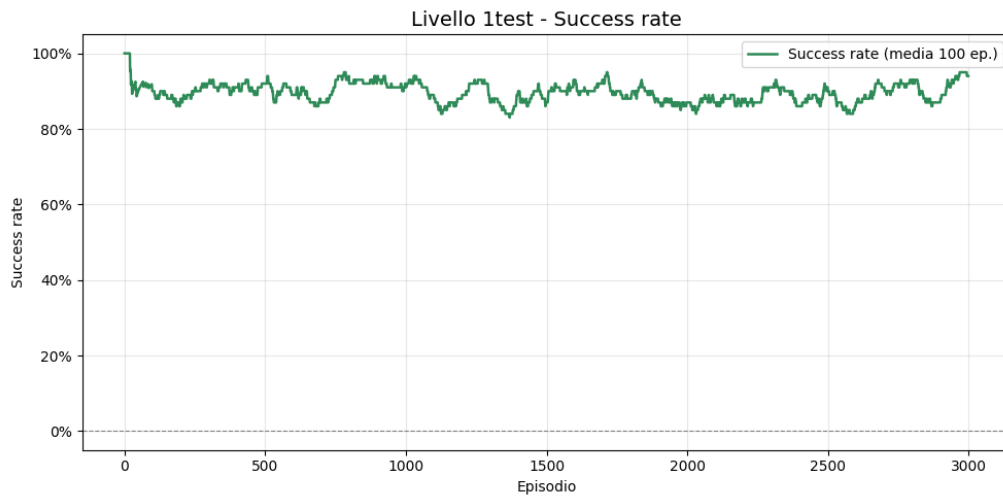


Figura 4.8: Success rate livello 1test

Il success rate si stabilizza già dai primi episodi attorno al 90%. La curva non mostra il picco iniziale basso tipico dei primi livelli del curriculum, né la fase di adattamento osservata nei livelli che partivano da zero. Il modello opera immediatamente in modo efficace sul nuovo ambiente, con oscillazioni contenute tra l’85% e il 95% per tutta la durata dei 3.000 episodi.

4.4.2 Steps per episodio

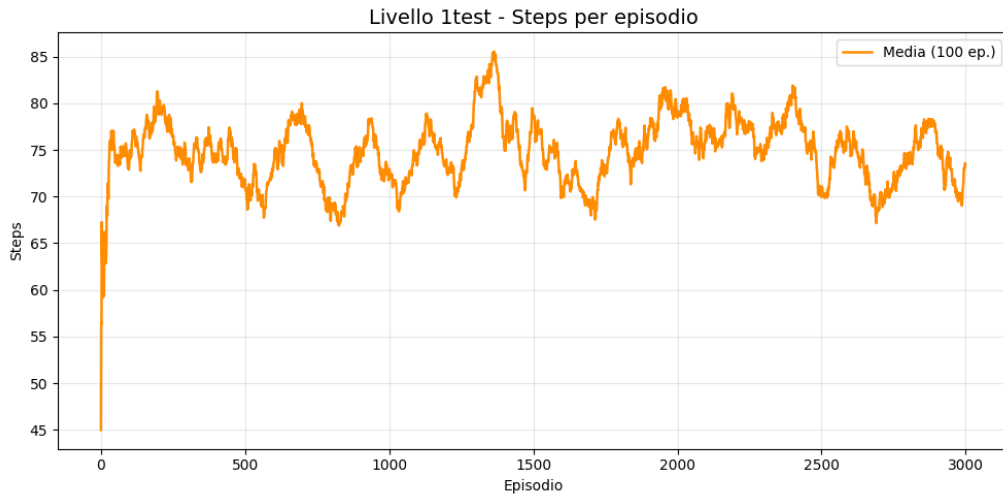


Figura 4.9: Steps per episodio livello 1test

Gli step medi si stabilizzano attorno a 75 passi per episodio, con oscillazioni tra 67 e 85. Il valore è più alto rispetto ai livelli della fase di training, in particolare rispetto al livello 1c che si attestava intorno a 40 step.

Questo comportamento può essere spiegato dal fatto che la griglia 17×17 è più grande, ci sono 5 fuochi da raggiungere invece di 4 e la densità di ostacoli è quasi doppia rispetto al livello 1c.

4.4.3 Distribuzione degli esiti

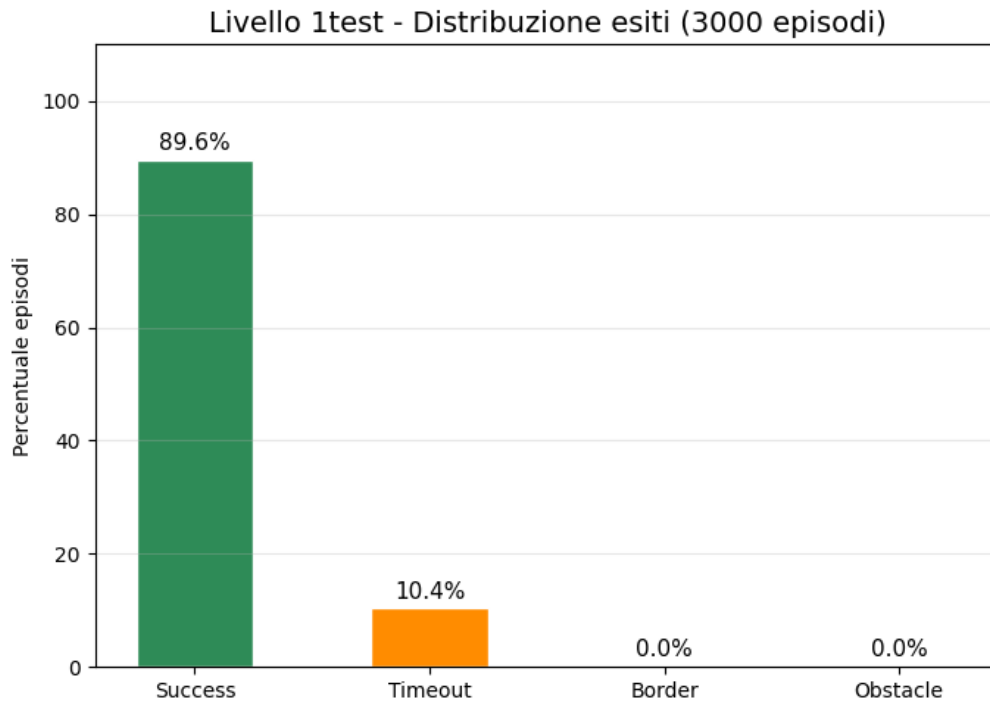


Figura 4.10: Distribuzione degli esiti livello 1test

Il dato più rilevante è la **totale assenza di collisioni**, né con i bordi né con gli ostacoli, nonostante la densità di ostacoli (20) sia maggiore rispetto al livello 1c. Questo indica che la capacità di evitare gli ostacoli e i bordi è stata appresa in modo robusto e si trasferisce anche in ambienti più complessi. I fallimenti sono dovuti esclusivamente a timeout (10.4%), ovvero a episodi in cui il drone non riesce a raggiungere tutti e 5 i fuochi entro i 150 passi.

Capitolo 5

Problema del Multi-Agent

5.1 Estensione al caso multi-agent

Dopo aver ottenuto risultati soddisfacenti nella fase single-agent, il progetto è stato esteso introducendo più droni all'interno della stessa griglia. L'obiettivo era verificare se più agenti potessero collaborare in modo efficace per spegnere i fuochi in meno tempo rispetto a un singolo drone, dividendosi il lavoro e coprendo aree diverse della griglia.

L'architettura scelta per il multi-agent è quella dell'**Independent Deep Q-Network (IDQN)**: ogni drone esegue il proprio ciclo di decisione in modo indipendente, ma tutti i droni condividono la stessa rete neurale e lo stesso replay buffer. Ogni drone osserva il proprio stato locale (che include le posizioni relative degli altri droni) e produce la propria azione in modo autonomo.

5.2 Risultati e limiti osservati

Nonostante l'architettura IDQN e il curriculum learning abbiano permesso al sistema di funzionare il comportamento osservato ha evidenziato un limite fondamentale: **i droni non si coordinano in modo efficace**.

Il problema principale è che, condividendo la stessa rete e ricevendo gli stessi tipi di reward, i droni tendono a convergere verso gli stessi fuochi invece di dividersi il lavoro, ignorando quelli più lontani.

Con una rete neurale condivisa e senza un meccanismo esplicito di comunicazione o coordinamento, ogni drone ottimizza la propria reward individuale seguendo la stessa politica degli altri. Il vettore di stato include la posizione relativa degli altri droni, ma questa informazione da sola non è sufficiente a generare una strategia cooperativa emergente: il modello apprende a evitare le collisioni fisiche con gli altri droni, ma non apprende a dividersi i fuochi in modo complementare.

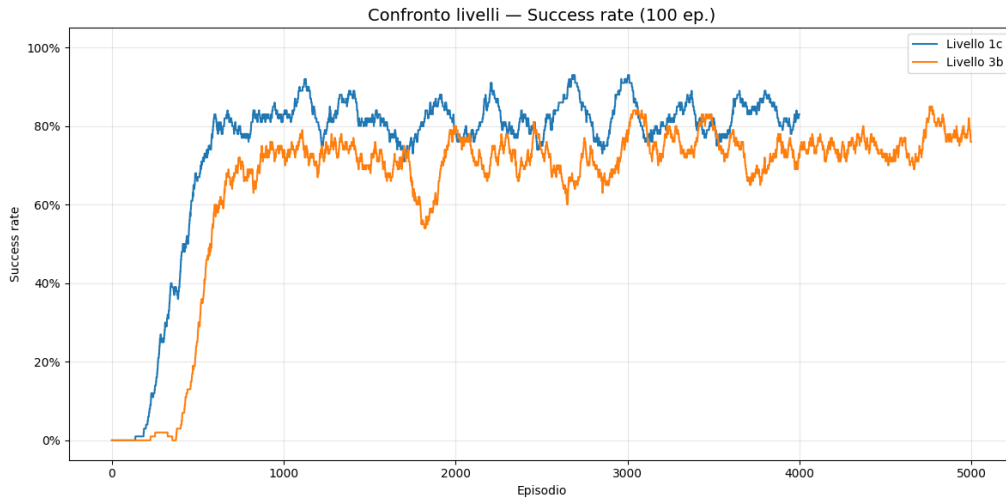


Figura 5.1: Confronto success rate 1c vs 3b.

Come si può notare dal grfico il livello 3b (arancione) si stabilizza intorno al 75–80%, un valore che rimane **leggermente inferiore** a quello del livello 1c (blu), che si stabilizza attorno all’80–90%. Il livello 3b, nonostante abbia tre droni contro uno solo del livello 1c, ci si aspetterebbe che più agenti possano completare il compito più facilmente. Il fatto che il success rate non migliori è la dimostrazione diretta che i tre droni non cooperano.

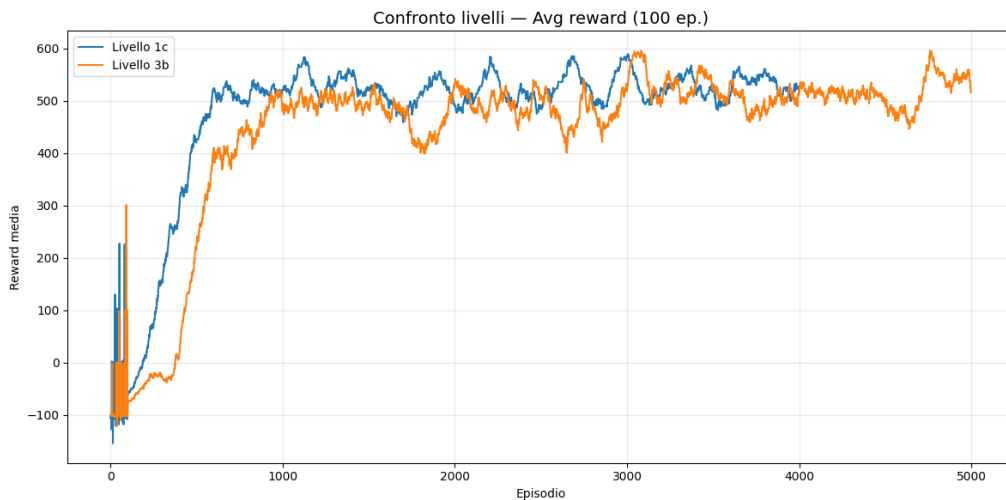


Figura 5.2: Confronto average reward 1c vs 3b.

Anche la reward media mostra che il livello 3b converge verso valori simili a quelli del livello 1c, con una convergenza quasi sovrapponibile nella fase stabile.

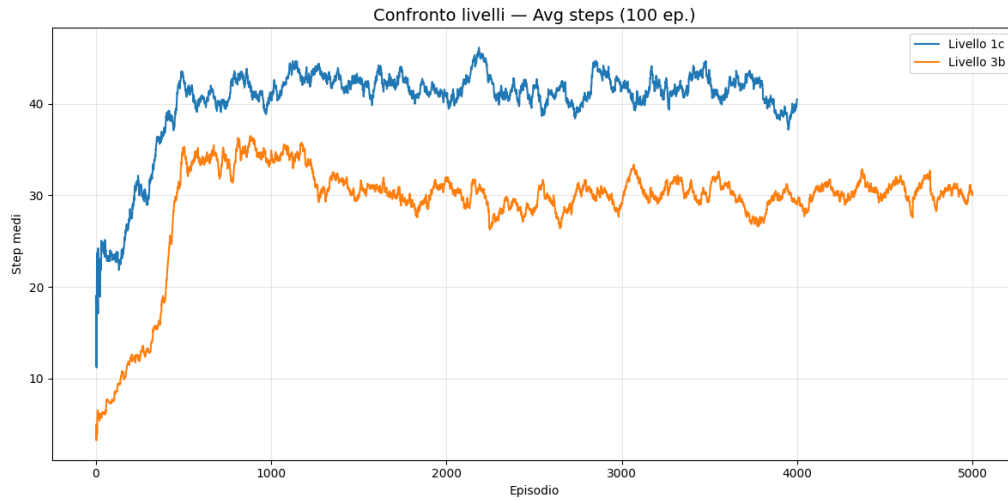


Figura 5.3: Confronto average steps 1c vs 3b.

Il grafico degli step medi mostra che il livello 3b impiega meno passi per episodio rispetto al livello 1c. Tuttavia questo non è necessariamente un segnale di coordinamento efficace. La griglia del livello 3b è più grande (18×18 vs 16×16) e ha più fuochi (5 vs 4), ma con tre droni la probabilità che almeno uno di essi raggiunga casualmente un fuoco vicino è statisticamente più alta. La riduzione degli step è quindi spiegabile in parte dalla maggiore copertura spaziale dei tre agenti, non da una divisione intenzionale del lavoro.

Capitolo 6

Test e modifiche sperimentali

Durante lo sviluppo del progetto sono state effettuate diverse modifiche sperimentali al sistema con l'obiettivo di migliorare le prestazioni del modello e in particolare di affrontare il problema del coordinamento nella fase multi-agent.

6.1 Modifiche alla rappresentazione dello stato

La rappresentazione dello stato è stata la variabile più esplorata durante la fase sperimentale. L'idea di base era che fornire al drone informazioni più utili potesse migliorare sia l'esplorazione individuale dell'ambiente sia il coordinamento tra più agenti.

Posizione assoluta dei fuochi (coordinate x, y)

Una prima variante testata ha sostituito le posizioni relative dei fuochi con le loro coordinate assolute nella griglia. L'ipotesi era che avere un riferimento fisso nello spazio potesse rendere più semplice per la rete apprendere traiettorie dirette verso gli obiettivi. Il risultato però non ha mostrato miglioramenti significativi rispetto alla versione con coordinate relative.

Distanza euclidea dai fuochi senza la direzione

Un'altra variante ha sostituito le coordinate relative dei fuochi con la sola distanza euclidea da ciascun fuoco, eliminando l'informazione sulla direzione. L'obiettivo era semplificare lo spazio di input riducendo il numero di valori necessari. Anche in questo caso i risultati non sono stati positivi, mostrando una convergenza più lenta.

Direzione degli altri droni

Nella fase multi-agent è stata testata l'aggiunta della direzione di movimento degli altri droni nello stato, oltre alla loro posizione relativa. L'idea era che conoscere non solo dove si trovano gli altri droni ma anche dove stanno andando potesse aiutare il modello a evitare sovrapposizioni e a dividersi i fuochi in modo più efficace assegnando reward negativa nel caso in cui due agenti si spostino verso la stessa direzione. Questa modifica si è però rivelata problematica per diversi motivi.

Il primo problema riguarda la natura stessa dell'algoritmo: ogni drone non ha un obiettivo fisso memorizzato, ma ad ogni passo calcola semplicemente l'azione migliore a

partire dallo stato corrente. Non esiste quindi una variabile che rappresenti “dove sta andando” il drone in senso strategico. L’unica informazione disponibile sulla direzione è quella del passo precedentemente eseguito, che non è necessariamente indicativa del comportamento futuro.

Il secondo problema riguarda la struttura del movimento: essendo limitato a quattro direzioni discrete, il drone si muove spesso in modo a zig-zag per raggiungere una destinazione in diagonale. La direzione dell’ultimo passo in questo caso è quasi casuale e non riflette la traiettoria complessiva verso il fuoco.

Infine, supponiamo che due droni debbano raggiungere due fuochi diversi ma entrambi posizionati nella stessa direzione rispetto alla loro posizione attuale. In questo scenario i due droni si muovono correttamente verso obiettivi distinti, ma la loro direzione è identica. Se la logica di reward penalizza i droni che si muovono nella stessa direzione, entrambi riceverebbero una penalità ingiustificata, premiando invece comportamenti in cui i droni si muovono in direzioni opposte anche quando questo non corrisponde a una vera divisione del lavoro.

Reward negativa se i droni stanno troppo vicini

Per scoraggiare i droni dal convergere nella stessa zona della griglia è stata introdotta una penalità applicata quando due droni si trovano entro una certa distanza l’uno dall’altro, anche senza collidere fisicamente. L’obiettivo era spingere i droni a mantenersi distanti e quindi a coprire aree diverse [7]. Nella pratica questa modifica ha prodotto un comportamento di fuga reciproca tra i droni, che tendevano a tenersi il più lontano possibile l’uno dall’altro indipendentemente dalla posizione dei fuochi.

Memorizzazione dei movimenti precedenti

Un ulteriore problema osservato durante le fasi di addestramento, indipendentemente dal numero di droni, riguardava la tendenza degli agenti a entrare in cicli di movimento ripetitivi. In particolare in prossimità di ostacoli o bordi, il drone percepiva il pericolo, retrocedeva, per poi rinavvicinarsi alla stessa cella e ripetere il comportamento indefinitamente.

Per affrontare questo problema è stata testata l’aggiunta allo stato di un vettore contenente le coordinate delle ultime quattro celle visitate dal drone, rappresentando una sorta di "coda" del suo percorso recente. Il drone riceveva una penalità quando si spostava su una cella già presente in questa coda, con l’obiettivo di scoraggiare la formazione di cicli brevi.

Questa modifica non ha prodotto i risultati attesi: i droni continuavano a formare cicli, semplicemente allungandoli fino a superare la lunghezza della coda memorizzata. Inoltre, nel caso multi-agent, memorizzare le coordinate delle code di tutti i droni e inserirle nello stato ha fatto crescere la dimensione del vettore di input, con tempi di addestramento eccessivamente elevati.

La soluzione adottata è stata quella di mantenere internamente una matrice di visita per ogni drone, ma di inserire nello stato solo un vettore compatto di 4 valori, corrispondenti al numero di volte che le quattro celle adiacenti sono state visitate nell’episodio corrente.

A ciascuna revisita viene assegnata una penalità negativa incrementale proporzionale al numero di visite precedenti sulla stessa cella, come descritto nella sezione 3.4.

Questo approccio si è rivelato più efficace e computazionalmente leggero, mantenendo la dimensione del vettore di stato moderato indipendentemente dal numero di droni.

Capitolo 7

Miglioramenti futuri

I miglioramenti descritti in questo capitolo non erano obiettivi primari del progetto, ma rappresentano sviluppi aggiuntivi che potrebbero essere esplorati in una fase successiva per estendere ulteriormente le capacità del sistema.

7.1 Coordinamento multi-agent con QMIX

Il limite più evidente del sistema attuale è l'assenza di coordinamento efficace tra i droni nella fase multi-agent.

Un'estensione possibile è l'adozione di **QMIX**, un algoritmo di Multi-Agent Reinforcement Learning che introduce una rete di mixing centralizzata durante il training. L'idea è che ogni drone mantenga la propria rete Q individuale per l'esecuzione, ma durante l'addestramento i valori Q individuali vengano combinati da una rete centrale che apprende a massimizzare la reward globale del team. In questo modo il coordinamento viene ottimizzato a livello di sistema senza richiedere comunicazione diretta tra i droni durante l'esecuzione.

Un'altra opzione è **VDN**, un approccio che frammenta il valore globale del team come somma dei valori individuali di ciascun agente. Rispetto a QMIX è un approccio più semplice e computazionalmente più leggero.

7.2 Divisione dei fuochi tramite algoritmi di assegnazione

Un approccio alternativo al problema del coordinamento consiste nell'introdurre un meccanismo esplicito di assegnazione degli obiettivi ai droni, indipendentemente dall'algoritmo di reinforcement learning utilizzato.

Una possibilità è l'utilizzo di **algoritmi di clustering** [6] [5]: prima di ogni episodio o periodicamente durante l'esecuzione, i fuochi vengono raggruppati in cluster e ciascun drone viene assegnato a un cluster diverso. In questo modo la divisione del lavoro è garantita e ciascun drone deve solo imparare a navigare verso i fuochi del proprio cluster.

Un'altra opzione è l'algoritmo di assegnazione basato sulle **ASTE** [2], che calcolano l'assegnazione drone-fuoco che minimizza la distanza totale da percorrere, massimizzando l'efficienza del team. Questo tipo di approccio è particolarmente utile quando il numero di droni e fuochi è elevato e la scelta di quale drone vada verso quale fuoco ha un impatto significativo sul tempo totale di completamento.

7.3 Ottimizzazione del percorso

Nella versione attuale i droni si muovono verso i fuochi seguendo la politica appresa dalla rete, che non garantisce necessariamente il percorso più breve. In presenza di ostacoli il drone può intraprendere percorsi non ottimali o aggirarli in modo inefficiente.

Un miglioramento possibile è l'integrazione di un algoritmo di **path planning** [1] per calcolare il percorso ottimale verso il fuoco assegnato, lasciando alla rete il compito di gestire le decisioni di alto livello (per esempio quale fuoco raggiungere, come coordinarsi con gli altri droni) mentre il movimento fisico viene gestito in modo deterministico ed efficiente.

7.4 Revisione e ottimizzazione dei livelli di curriculum

La struttura del curriculum attuale è stata progettata in modo incrementale durante lo sviluppo, ma potrebbe essere ottimizzata. Un possibile miglioramento riguarda la ricerca della configurazione di livelli che consenta di spingere il modello al massimo delle sue capacità.

Alcune variabili da esplorare potrebbero essere:

- **Dimensione della griglia:** verificare fino a che dimensione il modello riesce a generalizzare efficacemente, e se griglie molto grandi richiedono rappresentazioni dello stato diverse da quella attuale.
- **Numero di ostacoli e di fuochi:** aumentare progressivamente la densità di ostacoli e fuochi per valutare i limiti del sistema.

7.5 Aumento del livello di astrazione dell'ambiente

L'ambiente simulato attuale è volutamente semplificato per concentrarsi sull'apprendimento della politica di esplorazione. Molte delle semplificazioni adottate potrebbero essere rimosse in una versione più realistica del sistema.

Movimento a 360 gradi. Nella versione attuale i droni possono muoversi solo nelle quattro direzioni cardinali. Un sistema reale permetterebbe movimenti in qualsiasi direzione. Passare da 4 azioni discrete a un insieme continuo di direzioni richiederebbe una modifica sostanziale dell'architettura, eliminando il problema del movimento a zig-zag.

Altitudine variabile. Il modello attuale opera in uno spazio bidimensionale piatto in cui né i droni né gli ostacoli hanno un'altitudine. In un contesto reale sia gli ostacoli sia i droni operano in uno spazio tridimensionale, con la possibilità di sorvolare alcuni ostacoli invece di aggirarli.

Fuochi dinamici. Nel sistema attuale i fuochi sono statici: rimangono nella posizione in cui sono stati generati finché non vengono spenti. Un modello più realistico prevede che gli incendi si propaghino nel tempo, espandendosi alle celle adiacenti se non vengono raggiunti entro un certo numero di passi. Questo trasformerebbe il problema da statico a dinamico, richiedendo all'agente di tenere conto non solo di dove si trovano i fuochi ma anche di quanto tempo ha a disposizione prima che si espandano.

Gestione dell'acqua. Attualmente i droni spengono un fuoco semplicemente raggiungendo la sua cella, senza alcun costo legato alle risorse. In un contesto reale ogni

drone porta una riserva d'acqua limitata che si esaurisce dopo essere stata utilizzata e che dovrà essere rifornita.

Bibliografia

- [1] Ahmad Taher Azar et al. “Drone Deep Reinforcement Learning: A Review”. In: *Electronics* 10.9 (2021). ISSN: 2079-9292. DOI: 10.3390/electronics10090999. URL: <https://www.mdpi.com/2079-9292/10/9/999>.
- [2] Marc-Andr  Blais e Moulay A Akhloufi. “Drone swarm coordination using reinforcement learning for efficient wildfires fighting”. In: *SN Computer Science* 5.3 (2024), p. 314. URL: <https://link.springer.com/article/10.1007/s42979-024-02650-6>.
- [3] Alessandro Devo et al. “Autonomous Single-Image Drone Exploration With Deep Reinforcement Learning and Mixed Reality”. In: *IEEE Robotics and Automation Letters* 7.2 (2022), pp. 5031–5038. DOI: 10.1109/LRA.2022.3154019.
- [4] Ruipeng Tang et al. “Enhanced multi agent coordination algorithm for drone swarm patrolling in durian orchards”. In: *Scientific Reports* 15.1 (2025), p. 9139. URL: <https://www.nature.com/articles/s41598-025-88145-7>.
- [5] Reza Bairam Zadeh. “Multi-Agent System for Wildfire Management and Situational Awareness”. In: (gen. 2025). URL: https://dro.deakin.edu.au/articles/thesis/Multi-Agent_System_for_Wildfire_Management_and_Situational_Awareness/32110528.
- [6] Reza Bairam Zadeh et al. “A Multiagent Mission Coordination System for Continuous Situational Awareness of Bushfires”. In: *IEEE Transactions on Automation Science and Engineering* 20.2 (2023), pp. 1275–1291. DOI: 10.1109/TASE.2022.3183233.
- [7] Qingli Zeng e Farid Nait-Abdesselam. “Multi-Agent Reinforcement Learning-Based Extended Boid Modeling for Drone Swarms”. In: *ICC 2024 - IEEE International Conference on Communications*. 2024, pp. 1551–1556. DOI: 10.1109/ICC51166.2024.10622479.